

Rapport Individuel — Itération 1

IDE Java UML — Module Génération de Code

Nom : ESMAHI Salah eddine
Login : sei5471
Groupe : Groupe UML (Groupe 1)
Rôle : Conversion UML → Code Java
Date : Avril 2026

Contexte et objectif personnel

Dans le cadre de ce projet long en technologie objet, notre binôme est responsable du **module UML**, chargé de la conversion bidirectionnelle entre diagrammes de classes UML et code Java. Mon rôle spécifique au sein de ce binôme est la **génération de code Java à partir d'un modèle UML** (conversion UML → Java), tandis que mon binôme s'est chargé de la conversion inverse (Java → UML) ainsi que de l'affichage du diagramme en console.

Activités réalisées

Analyse et prise en main du modèle existant

En début d'itération, j'ai analysé l'ensemble des classes modèles produites par mon binôme : `JavaClass`, `JavaAttribut`, `JavaMethode`, `JavaParamettre`, `JavaRelation` et l'enum `Visibility`. Cette étape m'a permis de comprendre les conventions de nommage et les structures de données sur lesquelles allait reposer mon générateur.

✓ **Avancement : 100%**

Extension du modèle — `JavaClass`

Afin de permettre la génération correcte de classes abstraites et d'interfaces, j'ai ajouté les champs `isAbstract` et `isInterface` dans la classe `JavaClass`, avec leurs getters et setters associés, en coordination avec mon binôme pour ne pas introduire de régression dans son code.

✓ **Avancement : 100%**

Implémentation de `JavaCodeGenerator`

J'ai conçu et implémenté la classe `JavaCodeGenerator` qui constitue le cœur de ma contribution. Cette classe expose une méthode principale `generate(JavaClass)` qui construit le code Java correspondant via un `StringBuilder` et les sous-méthodes suivantes :

- `appendDeclaration` — génère la ligne `public [abstract] class / interface Nom`
- `appendAttributes` — génère les attributs avec leur visibilité
- `appendConstructors` — génère le constructeur vide et le constructeur complet
- `appendGettersSetters` — génère les getters et setters pour les attributs `private`
- `appendMethods` — génère les méthodes définies dans le modèle UML

✓ Avancement : 100%

Tests et validation

J'ai rédigé un fichier `TestCodeGenerator.java` indépendant de `Main.java` afin de tester mon générateur sans risquer de conflit avec le code de mon binôme. Les cas testés incluent : une classe simple avec attributs privés, la génération automatique des getters/setters et des constructeurs, ainsi qu'une classe avec méthodes paramétrées.

✓ Avancement : 100%

Bilan et difficultés rencontrées

La principale difficulté a été la **synchronisation avec mon binôme** sur les conventions du modèle (noms des getters, valeurs de l'enum `Visibility`), ce qui aurait pu rendre nos codes incompatibles. Nous avons résolu cela en communiquant en amont sur les interfaces avant de coder chacun de notre côté.

Une difficulté technique mineure a été la **version du compilateur Java** disponible, qui ne supportait pas la syntaxe moderne des *switch expressions*, ce qui m'a contraint à adapter ma méthode utilitaire `toKeyword`.

Reporté à l'itération 2

- Gestion des relations entre classes (`extends`, `implements`, composition, agrégation)
- Export du code généré vers un fichier `.java` sur le disque
- Intégration avec l'interface graphique JavaFX du module UML